Centro Universitário de Excelência
Sistemas de Informação

MecâniQA Nuvem – Time Vitoria

Autores: Aeverton Santos de Oliveira
Caio Victor Lopes Serra
Carlos Henrique Santos de Carvalho
Jonatas de Jesus Souza
Maciel Lopes Francisco
Natanael de Jesus Noronha

Itabuna, 2026

Agenda

Evolução do MecâniQA com implementação de monitoramento e observabilidade dos serviços, utilizando métricas e dashboards para acompanhar o comportamento da aplicação.

1. Contexto:

Objetivo e arquitetura de observabilidade.

2. Instrumentação IoT:

Python, prometheus_client e métricas.

3. Monitoramento:

Prometheus e quatro dashboards no Grafana.

4. Alertas e escalabilidade:

Quatro alertas, HPA e Metrics Server.

5. Testes e operação:

Teste de estresse, escalabilidade e recuperação.

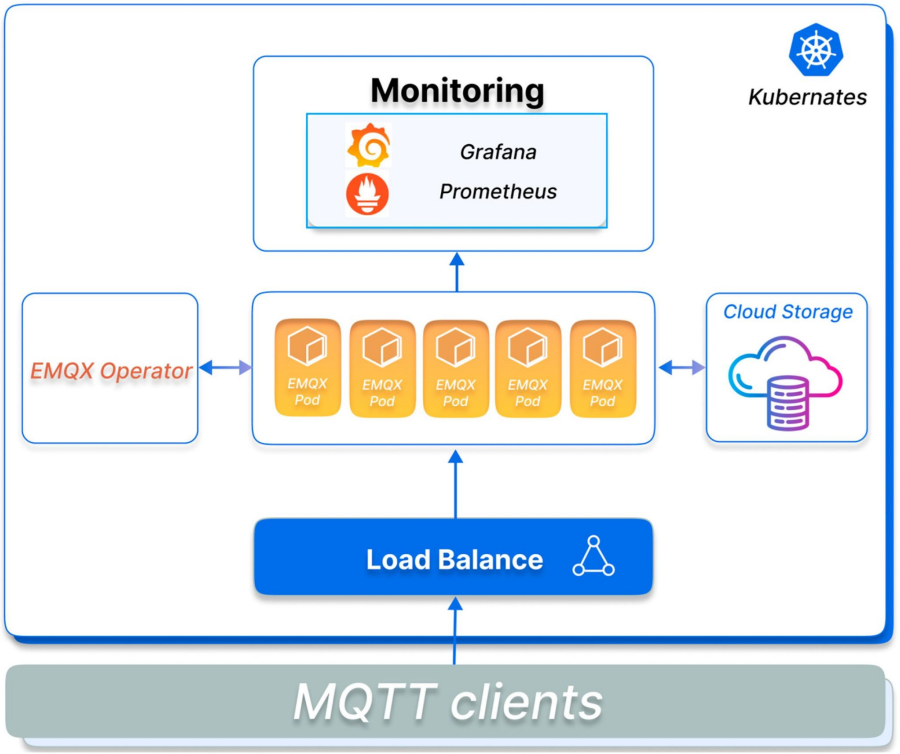
6. Gestão de serviços:

Runbooks, SLA, SLO, SLI, RTO e RPO.

Contexto do Problema

O desafio

- Falta de visibilidade sobre a saúde da aplicação.
- Picos de leituras IoT podem gerar sobrecarga.
- Identificar falhas antes da indisponibilidade.



Instrumentação IoT

Simulador Python instrumentado

- Biblioteca prometheus_client.
- Endpoint HTTP na porta 8000.
- Contadores de operações bem-sucedidas e com erro.
- Métricas de CPU e memória do processo.

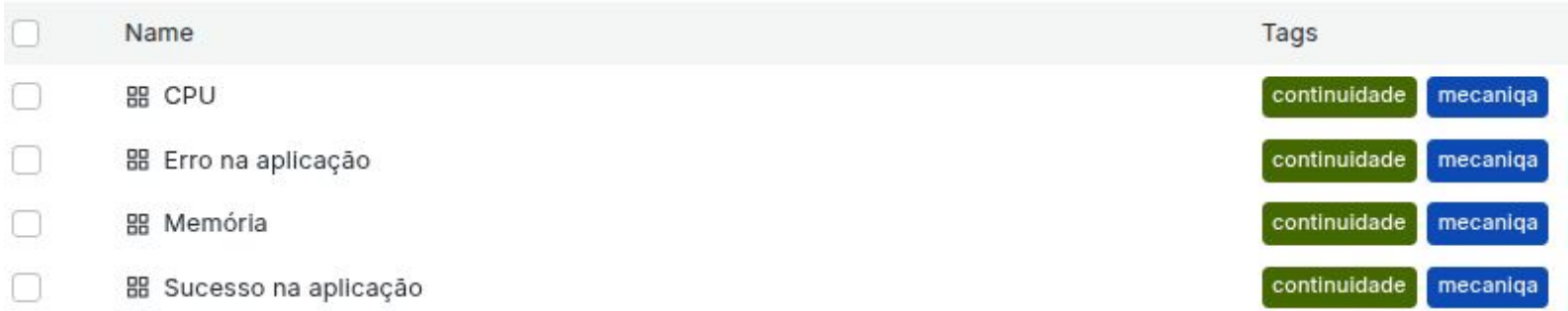
```python
from prometheus_client import Counter, Gauge, start_http_server


TEMPERATURE = Gauge(
    "iot_engine_temperature_celsius", "Temperatura simulada do motor em Celsius", ["sensor_id"]
)
OIL_PRESSURE = Gauge(
    "iot_oil_pressure_bar", "Pressao simulada do oleo em bar", ["sensor_id"]
)
VIBRATION = Gauge(
    "iot_vibration_meters_per_second_squared",
    "Aceleracao simulada de vibracao em metros por segundo ao quadrado",
    ["sensor_id"],
)
READINGS = Counter(
    "iot_sensor_readings_total", "Quantidade de leituras individuais geradas", ["sensor_id"]
)
LAST_UPDATE = Gauge(
    "iot_sensor_last_update_timestamp_seconds",
    "Horario Unix da ultima atualizacao do sensor",
    ["sensor_id"],
)
OPERATIONS = Counter(
    "iot_sensor_operations_total",
    "Tentativas de aquisicao por equipamento e resultado (falhas sinteticas)",
    ["sensor_id", "result"],
)
```

Monitoramento

- **Prometheus:** coleta periódica das métricas no endpoint /metrics.
- **Armazenamento:** registra as métricas como séries temporais.
- **Grafana:** utiliza o Prometheus como fonte de dados.
- **Dashboards:** quatro painéis para acompanhar CPU, memória, erros e operações bem-sucedidas.





Alertas e Escalabilidade

- **4 alertas no Grafana:** CPU, memória, erros e operações bem-sucedidas.
- **HPA:** escalabilidade automática dos Pods.
- **Metrics Server:** fornece métricas de utilização de recursos ao Kubernetes.
- **Configuração do HPA:** mínimo de 1 e máximo de 5 réplicas.



State	Name		Health	Summary	Next evaluation	Actions
Normal	CPU acima de 80% da capacidade reservada	Provisioned	ok	O consumo de CPU do simulador ultrapassou o limite definido.	in a few seconds	View More
Firing for 21m	Memória acima de 85% da capacidade reservada	Provisioned	ok	A memória utilizada pelo simulador ultrapassou o limite definido.	in a few seconds	View More
Normal	Taxa de erros acima de 20%	Provisioned	ok	A porcentagem de operações com erro ultrapassou 20%.	in a few seconds	View More
Normal	Taxa de sucesso abaixo de 80%	Provisioned	ok	A porcentagem de operações bem-sucedidas caiu abaixo de 80%.	in a few seconds	View More

```
maciel@User-PC:~/mecaniQA-nuvem-vitoria$ kubectl get hpa -n mecaniqa
NAME                REFERENCE                     TARGETS
MINPODS   MAXPODS   REPLICAS   AGE
iot-simulator-hpa   Deployment/iot-simulator   cpu: 151%/70%, memory: 61%/80%
1         5         5          7h44m
```

Testes e Operação

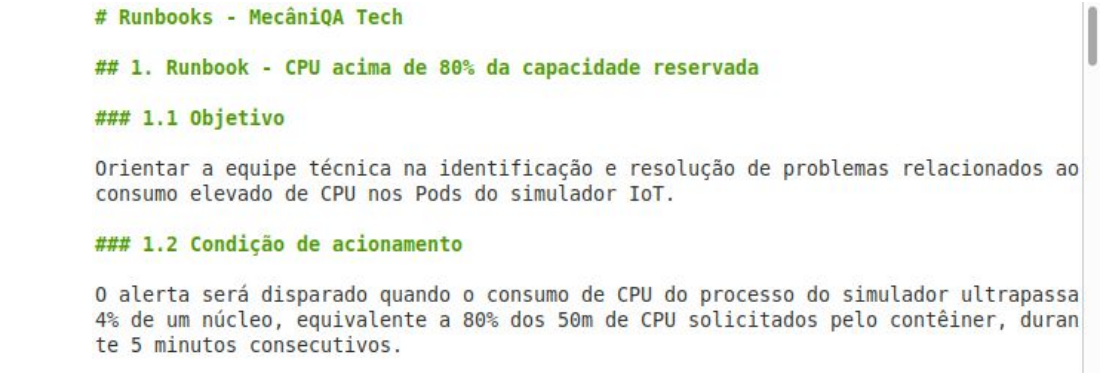
- **Teste de estresse:** simulação de aumento no volume de leituras IoT.
- **Escalabilidade:** aumento automático de 1 para 5 réplicas.
- **Recuperação:** redução das réplicas após normalização da carga.
- **Validação:** acompanhamento das métricas e dos alertas durante o teste.

```
maciel@User-PC:~/mecaniQA-nuvem-vitoria$ kubectl get hpa -n mecaniqa
NAME                REFERENCE                  TARGETS                         MI
NPODS   MAXPODS   REPLICAS   AGE
iot-simulator-hpa   Deployment/iot-simulator   cpu: 4%/70%, memory: 60%/80%   1
        5         3          9h
```

```
maciel@User-PC:~/mecaniQA-nuvem-vitoria$ kubectl get deployment iot-simulator -n
 mecaniqa
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
iot-simulator   5/5     5            5           35h
```

Gestão de Serviços

- **Runbooks:** quatro procedimentos para diagnóstico e resolução de alertas.
- **SLA:** disponibilidade de 99,9% para clientes Enterprise.
- **SLO e SLI:** metas e indicadores para acompanhar a qualidade do serviço.
- **RTO e RPO:** objetivos de recuperação e limites de perda de dados.



```
# Runbooks - MecâniQA Tech

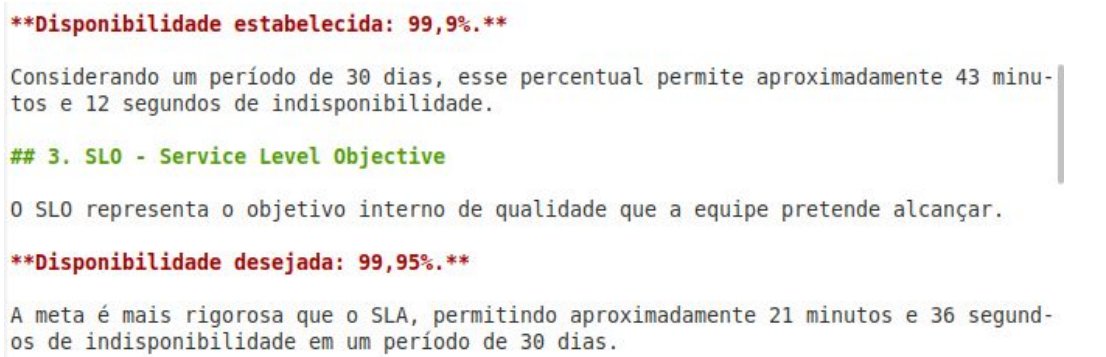
## 1. Runbook - CPU acima de 80% da capacidade reservada

### 1.1 Objetivo

Orientar a equipe técnica na identificação e resolução de problemas relacionados ao
consumo elevado de CPU nos Pods do simulador IoT.

### 1.2 Condição de acionamento

O alerta será disparado quando o consumo de CPU do processo do simulador ultrapassa
4% de um núcleo, equivalente a 80% dos 50m de CPU solicitados pelo contêiner, duran
te 5 minutos consecutivos.
```



```
**Disponibilidade estabelecida: 99,9%.**

Considerando um período de 30 dias, esse percentual permite aproximadamente 43 minu-
tos e 12 segundos de indisponibilidade.

## 3. SLO - Service Level Objective

O SLO representa o objetivo interno de qualidade que a equipe pretende alcançar.

**Disponibilidade desejada: 99,95%.**

A meta é mais rigorosa que o SLA, permitindo aproximadamente 21 minutos e 36 segund-
os de indisponibilidade em um período de 30 dias.
```

Considerações Finais

- **Visibilidade:** monitoramento centralizado das métricas da aplicação.
- **Resiliência:** alertas e escalabilidade automática para lidar com picos de carga.
- **Organização:** runbooks e indicadores para orientar a operação dos serviços.

“Mais visibilidade, controle e confiabilidade para a infraestrutura da MecâniQA.”



MecâniQA

Referências

- Prometheus - Documentação oficial: https://prometheus.io/docs/
- Prometheus Python Client - Documentação oficial: https://prometheus.github.io/client_python/
- Grafana - Documentação oficial: https://grafana.com/docs/grafana/latest/
- Kubernetes - Horizontal Pod Autoscaling: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
- Metrics Server - Documentação oficial: https://kubernetes-sigs.github.io/metrics-server/